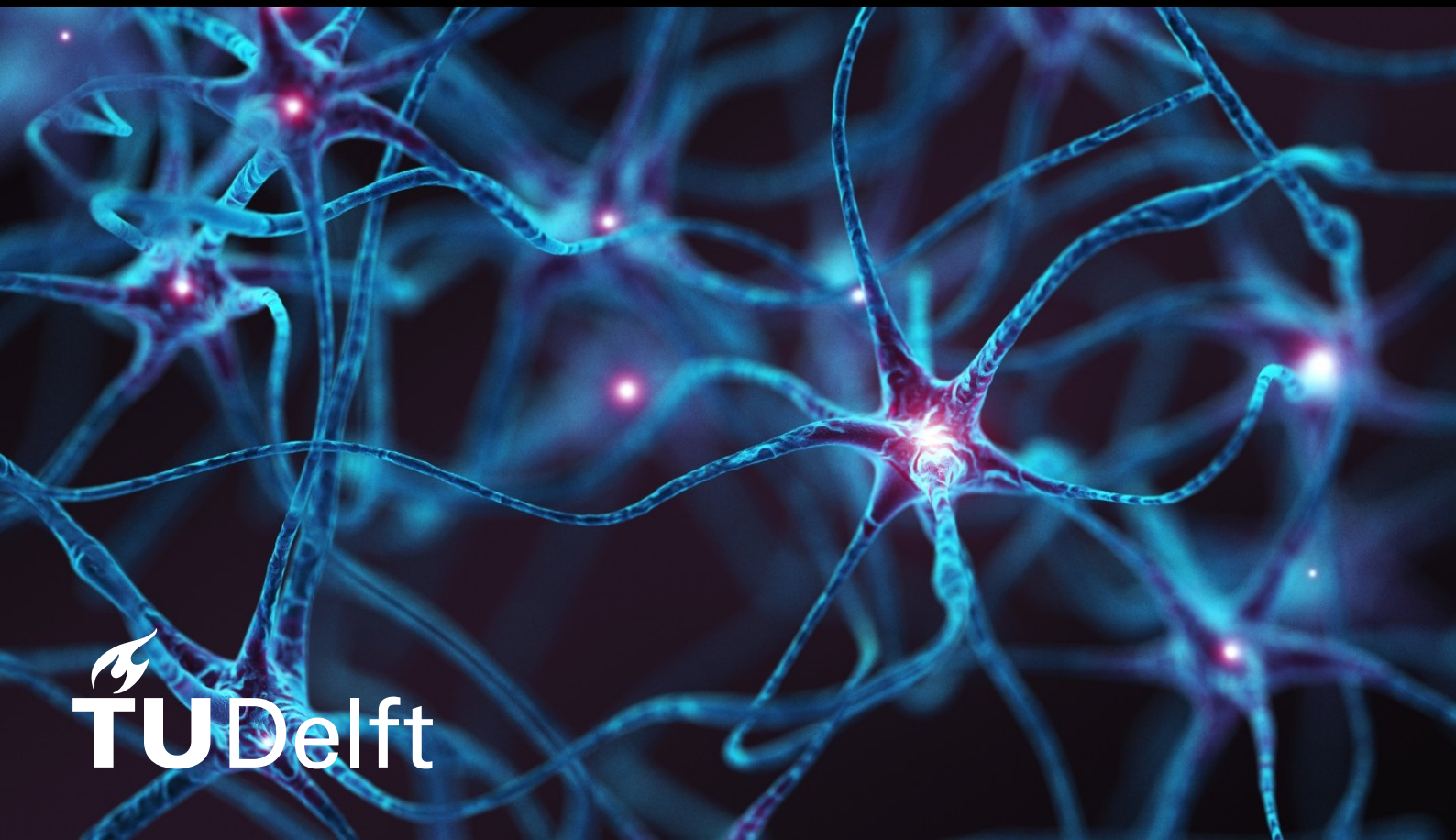
SNN for Spacecraft Docking

Investigation of Spiking Neural Networks Reliability

Bio-inspired Intelligence and learning for Aerospace Applications

Tommaso Lanzini



TU Delft

SNN for Spacecraft Docking

Investigation of Spiking Neural Networks Reliability

by

Tommaso Lanzini

6542301

Instructor:	Guido de Croon, Erik-Jan van Kampen
Project Duration:	June, 2026 - August, 2026
Faculty:	Faculty of Aerospace Engineering, Delft
GitHub:	https://github.com/tommasolanzini/SNNforDOCKING

Cover: Fraunhofer Institute for Integrated Circuits IIS

TU Delft

Abstract

Autonomous spacecraft proximity operations face optical challenges, including high-contrast illumination and motion blur, which degrade traditional frame-based sensors. To address this, this study presents a neuromorphic vision pipeline utilizing Dynamic Vision Sensor (DVS) coupled with a Spiking Neural Network (SNN) for continuous 6 Degrees of Freedom (DoF) pose estimation. Trained and evaluated on the Spiking Event-Based Neuromorphic Image Corpus (SEENIC) event-camera dataset, the SNN leverages an overlapping spatiotemporal stride and customized Arctangent (ATan) surrogate gradients to process sparse, asynchronous data. The architecture's performance is benchmarked against a structurally equivalent Artificial Neural Network (ANN). To simulate deployment in Low Earth Orbit (LEO), both networks are subjected to a Monte Carlo radiation simulator injecting Single Event Upset (SEU)s into the hardware memory. The results demonstrate that while the standard ANN experiences mathematical overflow from bit flips, the voltage-thresholding mechanics of the SNN provide higher fault tolerance, preventing system failure and maintaining spatial tracking under simulated radiation conditions.

Contents

Acronyms **iii**

0 Introduction . . . 1

1 Data Pipeline and Pre-Processing . . . 1

1.1 Limitations of Synthetic Event Generation . . . 1

1.2 Transition to Native Event Sensing . . . 2

1.3 Spatiotemporal Voxelization and Overlapping Stride . . . 2

1.4 Multi-Sequence Dataset Aggregation . . . 2

1.5 Ground Truth Synchronization and 6DoF Pose Interpolation . . . 3

2 Architecture . . . 3

2.1 Network Topology and Layer Specifications . . . 3

2.2 Sparsity Handling and Dropout Mechanics . . . 4

2.3 Surrogate Gradients and the Dead Neuron Problem . . . 4

2.4 Architectural Depth and Sequential Spike Attenuation . . . 5

2.5 The Continuous Output Translation . . . 5

3 Training . . . 6

3.1 Hardware Setup . . . 6

3.2 ANN-to-SNN Weight Transfer and Fine-Tuning . . . 6

3.3 Training Convergence and Transfer Learning Efficacy . . . 6

3.4 Learning Rate Optimization and Dynamic Scheduling . . . 7

3.5 Custom Weighted Loss . . . 7

3.6 Internal Temporal Smoothness Loss . . . 7

4 Results Analysis . . . 7

4.1 Statistical Evaluation across Multiple Runs . . . 7

4.2 Hyperparameter Sensitivity Analysis . . . 8

4.3 Real-Time Telemetry Visualization . . . 9

4.4 Fault Tolerance Against Cosmic Radiation . . . 9

4.5 Analysis of the Learned Strategy . . . 10

4.6 Limitations and Future Improvements . . . 10

References **11**

Acronyms

ANN	Artificial Neural Network
ATan	Arctangent
BER	Bit Error Rate
DoF	Degrees of Freedom
DVS	Dynamic Vision Sensor
GNC	Guidance Navigation and Control
GPU	Graphics Processing Unit
ISS	International Space Station
LEO	Low Earth Orbit
LIF	Leaky Integrate-and-Fire
MSE	Mean Squared Error
OOD	Out-of-Distribution
ReLU	Rectified Linear Unit
RMSE	Root Mean Square Error
SEENIC	Spiking Event-Based Neuromorphic Image Corpus
SEU	Single Event Upset
SNN	Spiking Neural Network

0. Introduction

Autonomous spacecraft rendezvous and proximity operations require precision, low latency, and reliability. Traditional frame-based optical sensors, while widely used, present limitations in the orbital environment. They are susceptible to motion blur during high-speed maneuvers and struggle with the dynamic lighting conditions of space, such as solar glare and shadows. These vulnerabilities can compromise the safety of docking procedures.

Neuromorphic engineering presents a biologically inspired alternative designed to overcome these limitations. Event cameras, or DVS, mimic the human retina by responding only to logarithmic changes in illumination. They offer a high dynamic range and microsecond-level temporal resolution while generating sparse data. Because they output asynchronous streams of events rather than dense static frames, they are an appropriate sensory input for SNN.

The primary objective of this project is to construct a robust SNN capable of performing continuous, 6 Degrees of Freedom (6DoF) pose estimation for spacecraft docking. To ensure the network learns physical dynamics rather than simulated artifacts, this research utilizes the SEENIC dataset, which provides native neuromorphic recordings of physical spacecraft approaches.

To evaluate the efficacy of the spiking architecture, its trajectory prediction performance is benchmarked against a standard ANN baseline.

Furthermore, deploying neural networks in LEO and beyond exposes the hardware to cosmic radiation. High-energy particles can cause SEUs, commonly known as bit flips, within the memory and processing units of the spacecraft's computer. Another objective of this research is to compare the fault tolerance of the developed SNN against the baseline ANN. By subjecting both networks to simulated radiation-induced bit flips, this study tests the hypothesis that the distributed, binary communication of spiking neurons provides higher resilience to hardware faults in space.

Finally, to ensure transparency and reproducibility, the methodology, including the neuromorphic data pipelines, custom loss functions, and network architectures, is open-source and available via GitHub.

1. Data Pipeline and Pre-Processing

This section describes the data handling and pre-processing pipeline, including the transition to native neuromorphic datasets, event voxelization, and telemetry synchronization.

1.1. Limitations of Synthetic Event Generation

Initial approaches to acquiring neuromorphic data for orbital rendezvous involved converting standard RGB broadcast videos, such as those recorded during docking procedures at the International Space Station (ISS), into synthetic event streams using simulators like `v2e` [1].

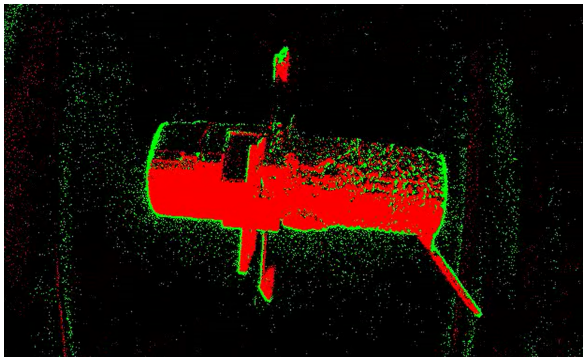


Figure 1: Neuromorphic frame visualization from the SEENIC dataset.

However, this method presents structural limitations. Simulated event streams retain the physical constraints of the source optical cameras, including limited dynamic range and susceptibility to motion blur. Furthermore, the algorithmic frame interpolation required to generate inter-frame events often introduces temporal artifacts.

Beyond these optical constraints, a primary limitation was the lack of ground truth telemetry. Standard ISS broadcast videos do not include pre-recorded 6DoF pose labels. Attempts to extract this spatial data using traditional visual tracking algorithms produced imprecise trajectories. Because supervised learning architectures require accurate labels to learn physical dynamics, using imprecise ground truth compromises model training.

To ensure accuracy, the methodology utilizes native Event Camera (DVS) datasets, specifically the SEENIC [2],

which provides raw asynchronous events and calibrated 6DoF pose annotations.

1.2. Transition to Native Event Sensing

Unlike frame-based sensors, a Dynamic Vision Sensor (DVS) responds to logarithmic changes in illumination, outputting an asynchronous stream of events [3]. An event is mathematically defined as a tuple $e_i = (x_i, y_i, t_i, p_i)$, where x_i and y_i denote the pixel coordinates, t_i is the microsecond-precise timestamp, and $p_i \in \{-1, 1\}$ represents the polarity of the brightness change.

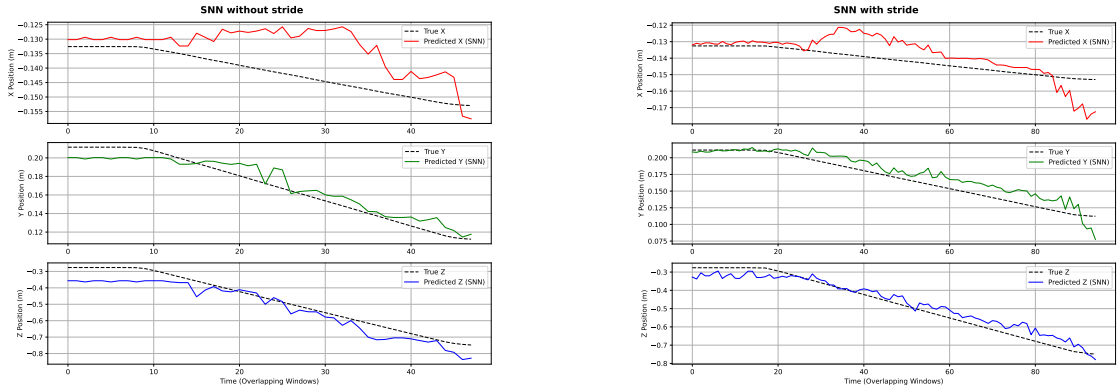
Utilizing native datasets eliminates the need for telemetry masking and background suppression. The high dynamic range (> 120 dB) of the physical DVS sensor handles orbital lighting conditions, such as solar glare and shadows, while the lack of a global shutter exposure time eliminates motion blur, preserving the geometric edges of the target spacecraft regardless of relative angular velocity.

1.3. Spatiotemporal Voxelization and Overlapping Stride

Standard ANN and SNN cannot directly process continuous, asynchronous event streams. The raw events must be accumulated into dense tensor representations.

To preserve the temporal dynamics required by the Leaky Integrate-and-Fire (LIF) neurons in the SNN, the continuous event stream is subdivided into discrete 50ms windows, formatted into a 4D tensor of shape $T \times C \times H \times W$ (where $T = 5$ micro-bins).

While processing discrete, non-overlapping blocks captures the general spatial trend, it restricts the temporal resolution and introduces artificial temporal disconnections between consecutive inferences. To improve trajectory fluidity, an **Overlapping Stride** methodology was implemented. A 50ms window is advanced by a 25ms stride, creating a 50% overlap between consecutive tensors. This doubles the resolution of the training and inference data and embeds temporal memory into the pipeline. By preventing the network's state from resetting abruptly, the overlapping stride allows the network to interpolate trajectories, reducing high-frequency jitter without saturating the LIF membrane potentials.



a) No Stride (50ms discrete windows)

b) Overlapping Stride (25ms advance)

Figure 2: Impact of the overlapping stride on spatial tracking. Without stride (a), the network successfully tracks the target but exhibits noticeable high-frequency jitter and lower temporal resolution. The overlapping stride (b) doubles the temporal resolution (from ~ 45 to > 90 windows), preserving temporal continuity and yielding a smoother tracking curve during the approach.

1.4. Multi-Sequence Dataset Aggregation

To prevent overfitting and ensure the network generalizes across varying lighting conditions and approach trajectories, a data-loading pipeline was implemented using different captures from the same dataset [2]. The training phase utilized four distinct physical capture scenarios: ambient lighting, direct center approaches, and targeted directional lighting. Utilizing PyTorch's `ConcatDataset` utility, these four sequences were unified into a randomized data loader.

To validate the network's inference capabilities, a fifth independent sequence was isolated. This sequence was

reserved for the testing phase to evaluate the model's ability to estimate 6DoF coordinates in unseen optical environments.

1.5. Ground Truth Synchronization and 6DoF Pose Interpolation

Supervised training of the network for proximity operations requires accurate ground truth labels. Unlike traditional 2D bounding boxes derived from standard visual tracking, the dataset used [2] provided 6DoF pose estimation, comprising the target's 3D translation coordinates and its spatial orientation.

Due to the asynchronous nature of the events, the continuous 50ms event windows rarely align perfectly with the discretely recorded timestamps of the ground truth hardware. To synchronize the data, a temporal interpolation algorithm was developed. For any given event window spanning from t_{start} to t_{end}, the exact temporal center t_{center} is queried, and linear interpolation is applied to the ground truth telemetry:

$$P(t_{\text{center}}) = P_0 + \frac{t_{\text{center}} - t_0}{t_1 - t_0}(P_1 - P_0) \tag{1}$$

Where P_0 and P_1 are the recorded positions at timestamps t_0 and t_1, respectively.

The interpolated vector $P \in \mathbb{R}^6$ encompasses the complete 6DoF state, structured as [X, Y, Z, R_x, R_y, R_z]. While standard aerospace kinematics use Spherical Linear Interpolation (SLERP) via quaternions to prevent geometric distortions, direct linear interpolation of the Euler angles was employed as an approximation. This simplification remains functional in this specific context due to the small angular variations occurring within the 50ms integration windows.

2. Architecture

This section describes the architecture of the 4-layer SNN developed for 6DoF pose estimation. It outlines the processing pipeline from spatiotemporal voxel ingestion to continuous coordinate regression, addressing event sparsity, signal attenuation, and surrogate gradient selection.

2.1. Network Topology and Layer Specifications

To extract spatial features from the high-dimensional event stream, a 4-layer feedforward spiking architecture was designed. As illustrated in Figure 3, the network is divided into a spatial feature extraction block (two convolutional layers) and a global regression block (two fully connected layers).

This separation ensures that local geometric patterns (such as the target spacecraft's structural edges and docking ring) are hierarchically processed before being flattened into a global spatial representation. The specifications of the architecture are detailed in Table 1.

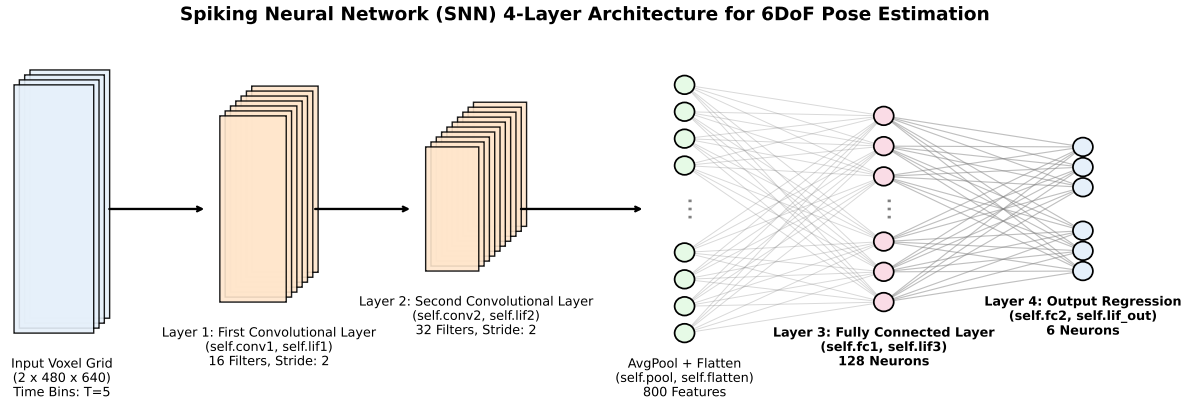


Figure 3: Block diagram of the 4-layer SNN architecture. The diagram illustrates the data flow from the spatiotemporal voxel grid through the convolutional layers, condensing into the fully connected block for the final continuous 6DoF coordinate regression.

Table 1: SNN Architecture and Layer Specifications

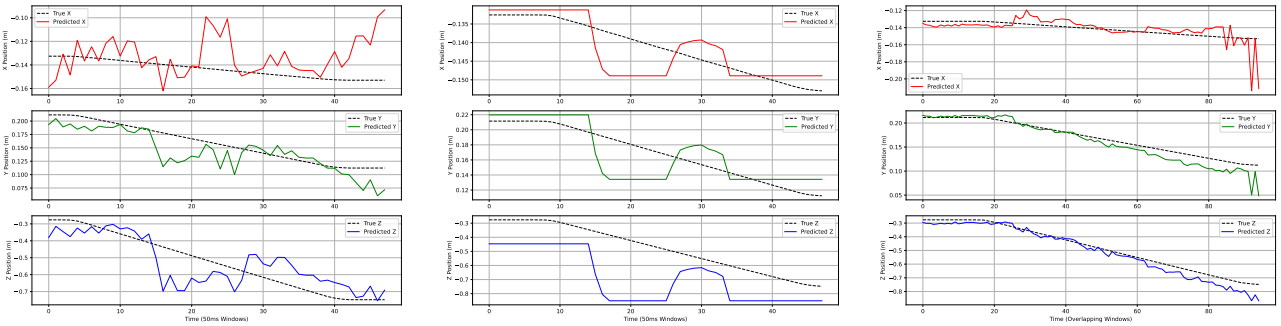
Layer Name	Specifications / Dimensions	Activation / Neuron Type
Input	Voxel Grid ($T = 5, C = 2, H = 480, W = 640$)	-
Layer 1 (Conv2D)	16 filters, 5×5 kernel, stride 2	LIF ($\beta = 0.95, U_{th} = 0.3$, ATan)
Layer 2 (Conv2D)	32 filters, 5×5 kernel, stride 2	LIF ($\beta = 0.95, U_{th} = 0.3$, ATan)
AvgPool	Adaptive 2D Pooling (5×5)	-
Flatten	Tensor flattening to 1D vector (800 Features)	-
Layer 3 (Dense)	Fully Connected Layer ($800 \rightarrow 128$)	LIF ($\beta = 0.95, U_{th} = 0.3$, ATan)
Dropout	Probability $p = 0.2$	-
Layer 4 (Output)	Regression Head ($128 \rightarrow 6$)	LIF (No reset, continuous U)

2.2. Sparsity Handling and Dropout Mechanics

Batch Normalization layers were omitted in this architecture. DVS event grids are inherently sparse; standard Batch Normalization treats the empty space (zeros) as a mean value, which amplifies sensor noise and saturates the LIF membrane potentials.

To enforce generalization, a Dropout layer was implemented within the 1D fully connected block, following the dense linear transformation and preceding the LIF spike generation. Initial optimization evaluated a standard dropout probability of $p = 0.5$. However, this rate is unsuitable for sparse neuromorphic data. Masking 50% of the active features induced voltage attenuation. The subsequent LIF neurons failed to accumulate sufficient membrane potential to reach their firing thresholds, halting forward signal propagation and degrading the network's predictions.

Consequently, the dropout rate was set to $p = 0.2$. As shown in Section 4, this rate balances regularization with spike retention. By zeroing out 20% of the flattened network activations during training, it prevents the SNN from over-relying on a single visual feature (such as solar glare) without attenuating the global regression block, resulting in stable tracking on unseen approach data.



a) No Dropout ($p = 0.0$)
Overfitting risk

b) Dropout ($p = 0.5$)
Voltage attenuation

c) Dropout ($p = 0.2$)
Optimal retention

Figure 4: Sensitivity analysis on the Dropout probability (p). Without dropout (a), the network risks overfitting on dominant features. A standard $p = 0.5$ (b) causes voltage attenuation and tracking errors in sparse DVS data. The optimized $p = 0.2$ (c) balances regularization with spike propagation.

2.3. Surrogate Gradients and the Dead Neuron Problem

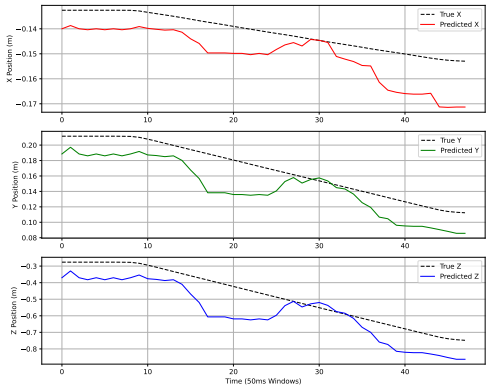
Training Spiking Neural Networks via backpropagation poses a mathematical challenge: spike generation is governed by a Heaviside step function, whose derivative is infinite at the threshold and zero everywhere else [4].

Initial training attempts utilizing a standard Sigmoid surrogate gradient resulted in a "staircase" effect in the predicted trajectories. Neurons whose membrane potentials fell slightly below the threshold received a gradient of zero, halting learning and creating "dead" neurons. To resolve this, the surrogate gradient was changed to an

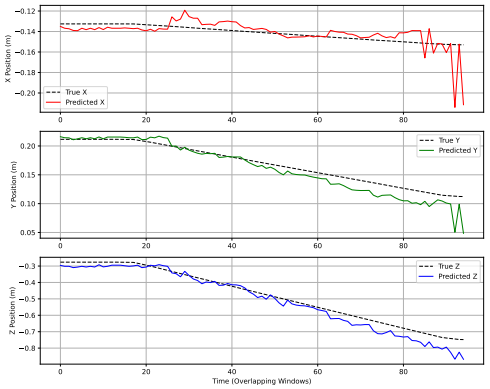
ATan function:

$$\sigma'(U) = \frac{1}{1 + (\alpha U)^2} \tag{2}$$

The ATan derivative ensures that sub-threshold neurons receive a non-zero gradient during backpropagation, preventing dead neurons and resulting in smoother spatial tracking.



a) Sigmoid Surrogate (Staircase Effect)



b) ATan Surrogate

Figure 5: Comparison of surrogate gradient functions. Using a standard Sigmoid derivative (a) causes dead neurons and a "staircase" effect in the predicted trajectory due to zero-gradients below the threshold. Using an ATan surrogate gradient (b) provides non-zero sub-threshold gradients, preventing dead neurons and resulting in continuous spatial tracking.

2.4. Architectural Depth and Sequential Spike Attenuation

To mitigate high-frequency jitter, an experiment was conducted to deepen the architecture by introducing a third convolutional block. However, as demonstrated in Figure 6, this induced sequential spike attenuation.

The sparse nature of the DVS data failed to generate sufficient membrane potential to overcome three consecutive LIF thresholds. This led to signal attenuation in low-activity temporal windows, resulting in coordinate drop-offs. As evident in the telemetry, the network manages initial tracking but loses spatial lock after window 75, exhibiting tracking errors across all axes and particularly on the Y and Z coordinates.

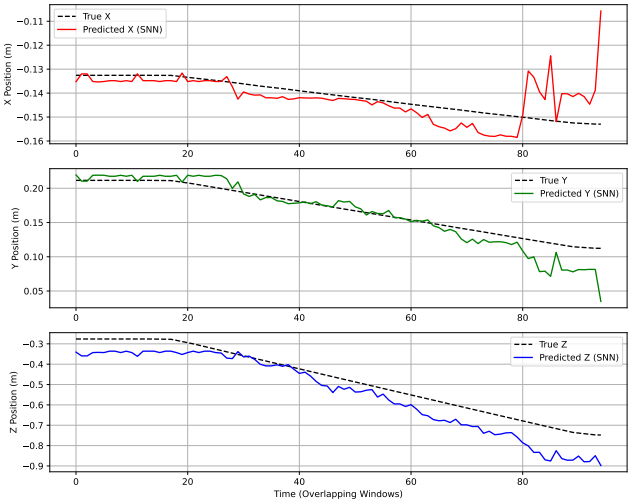


Figure 6: Trajectory errors caused by sequential spike attenuation when adding a 3rd convolutional layer, exhibiting tracking loss during the final approach.

While the overall translation Root Mean Square Error (RMSE) for this configuration evaluates to 0.0454 m, this global average masks localized tracking failures. The Z-axis exhibits the highest signal degradation (Mean Absolute Error of 0.0653 m), indicating that for sparse spatiotemporal voxel grids, a shallower architecture limits signal attenuation and preserves geometric integrity more effectively than deeper topologies.

2.5. The Continuous Output Translation

The final linear regression head maps the global features into 6 continuous variables (X, Y, Z, R_x, R_y, R_z). In the SNN implementation, spatial coordinates cannot be represented by discrete, binary spikes (0 or 1). Consequently, for the final output layer (`lif_out`), binary spike generation is bypassed. The network utilizes the accumulated membrane potential ($U[t]$) of the final 6 neurons as the continuous floating-point prediction.

3. Training

This section describes the training methodology and optimization strategies employed for the neuromorphic architecture. It outlines the ANN-to-SNN transfer learning paradigm used to bypass random initialization, the custom pose-weighted loss function for balancing translational and rotational errors, and the dynamic learning rate scheduling.

3.1. Hardware Setup

To ensure efficient processing of the 4D spatiotemporal tensors, the pipeline was optimized for Graphics Processing Unit (GPU) acceleration. The dataset loaders, surrogate gradients, and network architecture were integrated with the NVIDIA CUDA API via PyTorch [5]. This hardware acceleration allowed for parallelized batch processing of the overlapping event windows, which was required to handle the high-dimensional neuromorphic data during the training loop.

3.2. ANN-to-SNN Weight Transfer and Fine-Tuning

Spiking Neural Networks present training challenges due to their discrete, non-differentiable nature. Specifically, LIF neurons are sensitive to random weight initialization [6]. An unfavorable initialization can lead to voltage attenuation or trap the optimizer in local minima, causing tracking errors.

To bypass this initialization, an ANN with an isomorphic (structurally identical) topology was developed. Because continuous ANNs do not have discrete thresholding constraints, standard backpropagation maps the geometric feature extractors for 6DoF pose estimation.

Once the ANN converged, a Transfer Learning paradigm was employed. The trained weight matrices from the ANN were transferred to the SNN architecture. Following this transfer, the SNN underwent a fine-tuning phase. The Adam optimizer's learning rate was reduced by an order of magnitude (from 0.001 to 0.0001). This allowed the continuous weights to adapt to the LIF thresholding mechanics without losing the inherited spatial representations.

3.3. Training Convergence and Transfer Learning Efficacy

The effect of the Transfer Learning paradigm and Custom Weighted Loss is reflected in the network's training dynamics.

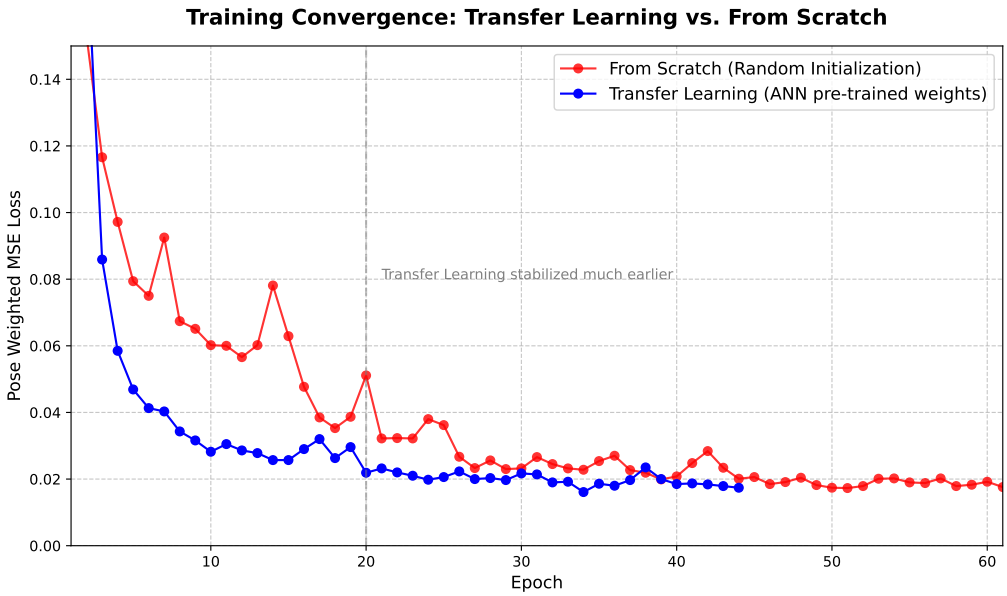


Figure 7: Comparison of the Custom Pose Weighted Mean Squared Error (MSE) Loss progression over 60 epochs. The Transfer Learning approach (blue) demonstrates stable convergence, whereas training from scratch (red) exhibits instability and error spikes.

Figure 7 compares the progression of the Pose Weighted MSE Loss during the SNN fine-tuning phase against a

baseline SNN trained from scratch with random weight initialization. Because the transfer learning network was initialized with pre-trained continuous weights, it bypasses the exploratory phase. As observed, the transfer learning curve (blue) exhibits an initial drop, stabilizing earlier than the baseline.

In contrast, the network trained from scratch (red) exhibits instability with random initialization. Its progression remains at a higher loss state for the first 20 epochs, with error spikes observed around epochs 7, 14, and 20. This comparison indicates that the ANN-to-SNN weight initialization prevents the optimizer from settling in local minima and reduces the computational epochs required to reach convergence.

3.4. Learning Rate Optimization and Dynamic Scheduling

To achieve stable convergence and avoid overshooting the thresholds of the LIF neurons, the learning rate was adjusted dynamically. The network was optimized utilizing the Adam optimizer. To refine the synaptic weight updates as the network approached optimal spatial tracking, a `ReduceLROnPlateau` scheduler was implemented.

The scheduler monitored the running loss, decaying the learning rate by 50% (factor = 0.5) if no improvement was detected for 3 consecutive epochs. Coupled with an early-stopping mechanism (triggered after 10 epochs of stagnation), this dynamic scheduling allowed larger updates during initial training and transitioned to smaller adjustments during convergence, preserving the learned feature representations.

3.5. Custom Weighted Loss

Standard MSE treats translation errors (measured in meters) and rotational errors (measured in radians) equally. Due to the small scale of the positional variations in the dataset (e.g., 0.15 meters), a standard loss function causes optimization imbalances, leading the network to optimize for angular deviations over positional tracking. A `PoseWeightedMSELoss` function [7] was implemented to decouple these metrics, assigning a 10.0× weight to translation and 1.0× to rotation, prioritizing spatial positioning while maintaining orientation.

3.6. Internal Temporal Smoothness Loss

To reduce spatial discontinuities between consecutive frames, a Temporal Smoothness penalty was introduced. Because the `DataLoader` shuffles external 50ms windows to prevent overfitting, external continuity cannot be measured during training.

Instead, the loss function evaluates the internal membrane potentials of the network across the $T = 5$ micro-bins within a single window. By squaring and penalizing the membrane potential differences between consecutive micro-bins (ΔU^2), the network maintains a stable voltage throughout the integration window, resulting in smoother trajectory inferences.

4. Results Analysis

This section presents the evaluation and performance metrics of the SNN architecture. It provides a statistical analysis of the network's spatial tracking uncertainty, assesses its sensitivity to hyperparameter variations, and concludes with a simulated hardware validation evaluating the SNN's fault tolerance against radiation-induced bit flips.

4.1. Statistical Evaluation across Multiple Runs

To quantify robustness, the 4-layer SNN architecture was evaluated across $N = 10$ independent fine-tuning runs. Each instance was initialized utilizing the ANN-to-SNN transfer learning paradigm to establish a consistent baseline before exposing the network to stochastic dropout and dataset shuffling.

Figure 8 presents the mean predicted trajectory (μ) alongside its corresponding uncertainty bounds ($\pm 1\sigma$), and aggregates the error distributions.

To provide an assessment of the network's spatial tracking, the MSE and RMSE were calculated for each of the 6 Degrees of Freedom, as detailed in Table 2.

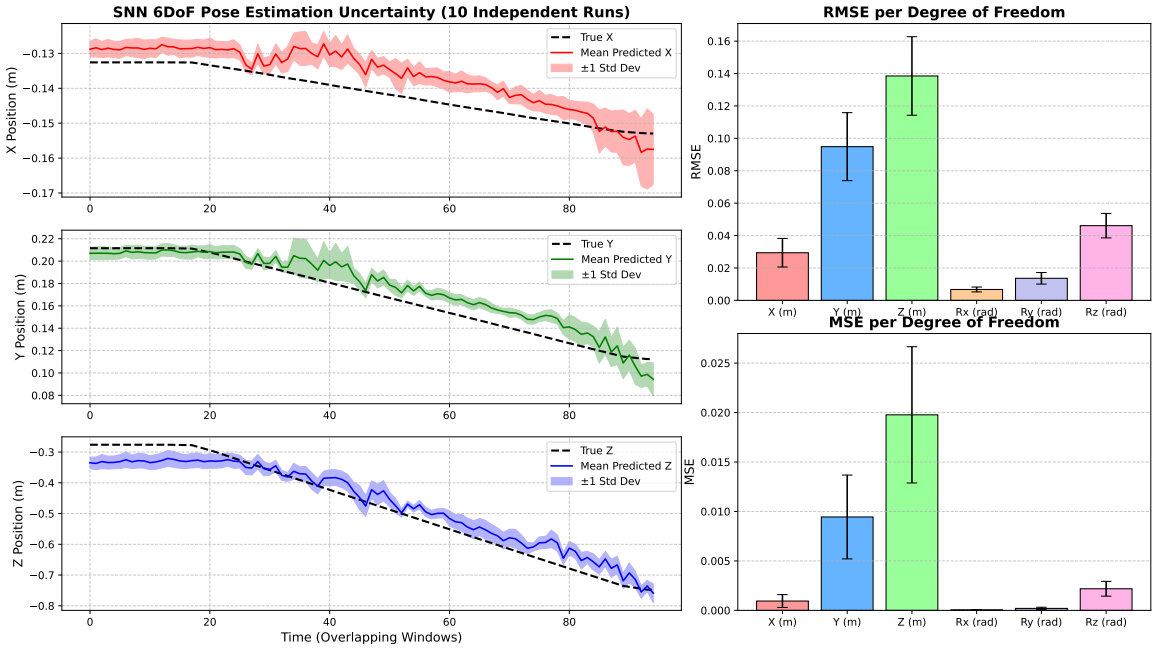


Figure 8: Statistical evaluation across 10 independent SNN fine-tuning runs. Left: Mean spatial trajectory and uncertainty bounds (±1 Standard Deviation). Right: RMSE and MSE distributions per Degree of Freedom.

Table 2: Error Metrics per Degree of Freedom across 10 Independent Runs ($\mu \pm 1\sigma$)

Degree of Freedom	RMSE ($\mu \pm \sigma$)	MSE ($\mu \pm \sigma$)
X Translation (m)	0.0294 ± 0.0088	0.0009 ± 0.0007
Y Translation (m)	0.0949 ± 0.0210	0.0094 ± 0.0042
Z Translation (m)	0.1385 ± 0.0242	0.0198 ± 0.0069
Rx Rotation (rad)	0.0067 ± 0.0015	$< 0.0001 \pm 0.0000$
Ry Rotation (rad)	0.0136 ± 0.0036	0.0002 ± 0.0001
Rz Rotation (rad)	0.0461 ± 0.0075	0.0022 ± 0.0007

The empirical metrics demonstrate structural stability. Depth estimation (Z-axis) presents a geometric challenge, exhibiting the highest mean RMSE (0.1385 m). Nevertheless, the spatial uncertainty (σ) on this axis remains bounded at 0.0242 m. The network's rotational predictions yield pitch (Ry) and roll (Rx) errors under 0.015 rad.

The SNN maintains spatial tracking down to the final proximity windows. Unlike standard ANN baselines, which exhibit Out-of-Distribution (OOD) errors when the spacecraft fills the camera's Field of View, the LIF neurons act as signal limiters. By spiking and resetting at their membrane threshold, they filter out high pixel intensity variations, demonstrating the architecture's suitability for spaceflight operations.

4.2. Hyperparameter Sensitivity Analysis

A sensitivity analysis was conducted to evaluate the impact of the LIF membrane threshold and the Surrogate Gradient steepness (α).

- **Threshold Sensitivity:** As illustrated in Figure 9, varying the threshold between 0.3, 0.6, and 1.0 revealed a non-linear relationship with tracking accuracy. While the 0.3 threshold balanced noise rejection with signal propagation, higher thresholds impaired the learning process. Models constrained by thresholds of 0.6 and 1.0 suffered from voltage attenuation. Failing to propagate sufficient spikes during backpropagation, they could not map the spatial geometry; this manifests during testing as trajectory errors (evident in the 0.6 model) and high-frequency jitter with spatial overshooting (prominent in the 1.0 model).
- **Surrogate Gradient Impact:** Replacing the standard Sigmoid derivative with the ATan function ($\alpha = 2.0$) accelerated convergence by providing broader gradients to sub-threshold neurons, confirming the utility of

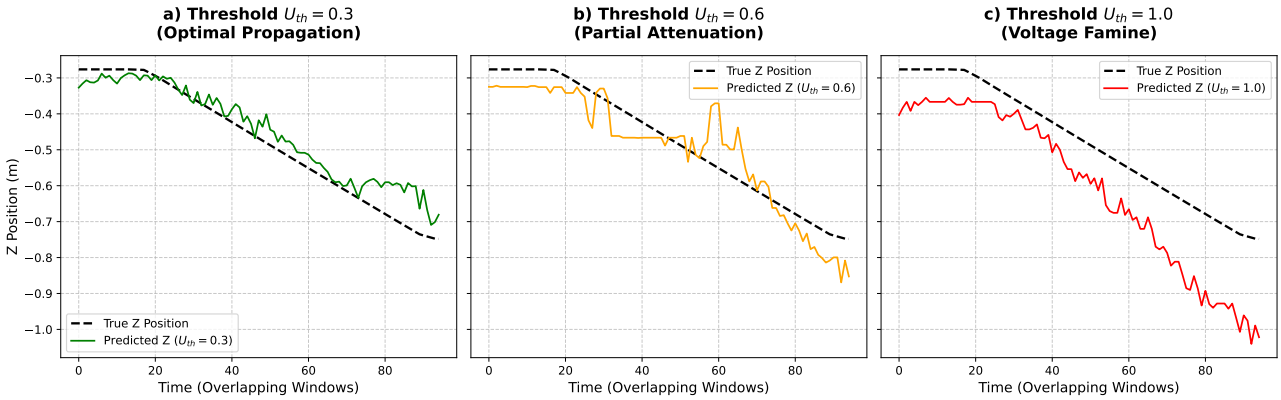


Figure 9: Sensitivity analysis of the LIF membrane threshold on the Z-axis predictions. The 0.3 threshold (a) allows spike propagation for accurate tracking. Increasing the threshold to 0.6 (b) introduces attenuation and mid-approach errors. A threshold of 1.0 (c) progressively starves the network of voltage during training. This prevents the architecture from learning geometric features, leading to signal degradation, high-frequency jitter, and spatial overshooting during the final approach phase.

surrogate derivatives in sparse event processing (2.3).

4.3. Real-Time Telemetry Visualization

To visualize the network's spatial tracking, a simulated telemetry dashboard was developed. As observed at https://github.com/tommasolanzini/SNNforDOCKING/blob/main/snn_real_telemetry.gif, the dashboard synchronously visualizes the raw DVS event stream (50ms temporal windows), the real-time 3D trajectory tracking, and the continuous RMSE. This visualization confirms that the SNN integrates asynchronous neuromorphic data to maintain a 6DoF estimation during the docking phase.

4.4. Fault Tolerance Against Cosmic Radiation

Deploying neural networks in LEO exposes hardware to cosmic radiation, where high-energy particles cause Single Event Upsets (SEUs), or bit flips, within the spacecraft's memory [8]. To test the hypothesis that spiking neurons provide fault resilience, a Monte Carlo simulator injected random bit flips into both the trained SNN and a baseline ANN across varying Bit Error Rate (BER). To ensure a direct comparison, the ANN utilizes an isomorphic topology to the SNN and demonstrates similar baseline precision before fault injection (at BER 0), as evident in Figure 10.

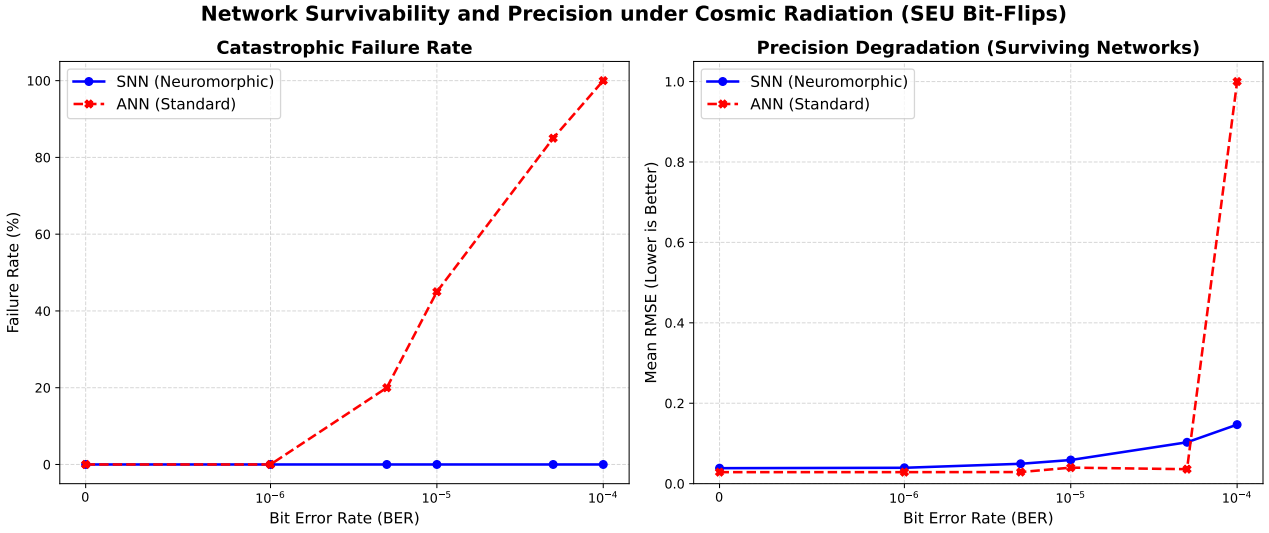


Figure 10: Network survivability and precision under simulated cosmic radiation (Monte Carlo SEU injection). Left: Failure rate as a function of BER. Right: Precision degradation (Mean RMSE) of the surviving network inferences.

As shown in Figure 10 (left), the continuous-valued ANN is vulnerable, suffering a 100% failure rate at a BER of 10^{-4}. When a bit flip corrupts a weight's exponent, the ANN exponentially magnifies the error through continuous activations (Rectified Linear Unit (ReLU)), causing trajectory overflows. Conversely, the SNN maintains structural survivability (0% failure rate).

Evaluating the surviving inferences (Figure 10, right) highlights the SNN's degradation profile. While its Mean RMSE increases to approximately 0.15, the ANN exhibits an exponential error increase prior to failure. The SNN's resilience stems from LIF mechanics: a neuron can only output a discrete, binary spike (1). This signal clipping isolates hardware faults locally, preventing cascading errors and maintaining spatial tracking under simulated orbital conditions.

4.5. Analysis of the Learned Strategy

Analysis of the network's internal activations reveals that the SNN solves the pose estimation task by tracking specific geometric features rather than global intensity changes. The early convolutional layers function as spatiotemporal edge detectors, exhibiting high firing rates at structural boundaries, specifically the linear edges of the solar arrays and the circular docking ring. Background asynchronous noise is systematically suppressed by the LIF membrane thresholding. The global regression block infers monocular depth (Z-axis) by evaluating spatial divergence. As the target spacecraft approaches, the expanding event-clusters of the solar panels activate a progressively wider distribution of neurons in the flattened layers; the network maps this feature spread directly to proximity. Simultaneously, rotational orientation (R_x, R_y, R_z) is derived by tracking the geometric skew and elliptical deformation of the docking ring across the sensor's field of view.
Finally, the LIF membrane potentials sustain these structural representations across the overlapping integration windows. This temporal integration prevents tracking drop-offs during moments of relative target stasis, when the generation of new polarity events is minimal, maintaining a stable 6DoF estimation throughout the final approach phase.

4.6. Limitations and Future Improvements

Despite the demonstrated performance, architectural limitations remain. Currently, the network's temporal receptive field is bounded by the 50ms integration window and the 25ms overlapping stride. Consequently, the feedforward architecture lacks long-term temporal memory, meaning it cannot fully leverage the historical trajectory of the approach to predict future states. This limitation is the reason why explicit temporal smoothness penalties were ineffective during training, as forcing voltage continuity across discrete windows attenuated the surrogate gradients.

Future iterations will address this structural limitation by transitioning to a Recurrent SNN. Incorporating recurrent synaptic connections or Adaptive LIF neurons would enable the network to maintain a kinematic memory of the spacecraft's trajectory, eliminating the need for dataset overlapping. Furthermore, to address the residual high-frequency jitter in the spatial predictions, future research will explore coupling the SNN regression head with a discrete Kalman filter. This would fuse the neural predictions with a kinematic motion model, providing a mechanism to smooth the 6DoF telemetry before it is fed into the spacecraft's Guidance Navigation and Control (GNC) loops.

References

[1] Yuhuang Hu, Shih-Chii Liu, and Tobi Delbruck. "v2e: From video frames to realistic DVS events". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*. 2021, pp. 1312–1321. DOI: 10.1109/CVPRW53098.2021.00146. URL: https://doi.org/10.1109/CVPRW53098.2021.00146.

[2] Ethan Elms et al. *SEENIC: dataset for Spacecraft posE Estimation with NeuromorphIC vision*. Version 1.1. Zenodo, Oct. 2022. DOI: 10.5281/zenodo.7370076. URL: https://doi.org/10.5281/zenodo.7370076.

[3] Guillermo Gallego et al. "Event-based vision: A survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.1 (2022), pp. 154–180. DOI: 10.1109/TPAMI.2020.3008413. URL: https://doi.org/10.1109/TPAMI.2020.3008413.

[4] Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks". In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 51–63. DOI: 10.1109/MSP.2019.2931595. URL: https://doi.org/10.1109/MSP.2019.2931595.

[5] Jason K Eshraghian et al. "Training spiking neural networks using lessons from deep learning". In: *Proceedings of the IEEE* 111.9 (2023), pp. 1015–1054. DOI: 10.1109/JPROC.2023.3308088. URL: https://doi.org/10.1109/JPROC.2023.3308088.

[6] Bodo Rueckauer et al. "Conversion of continuous-valued deep networks to spiking neural networks". In: *Frontiers in Neuroscience* 11 (2017), p. 682. DOI: 10.3389/fnins.2017.00682. URL: https://doi.org/10.3389/fnins.2017.00682.

[7] Tommaso Lanzini. *SNNforDOCKING: Spiking Neural Networks for Spacecraft Docking*. https://github.com/tommasolanzini/SNNforDOCKING. 2026. URL: https://github.com/tommasolanzini/SNNforDOCKING.

[8] Alvaro Castañeda et al. "Fault tolerance in spiking neural networks for edge computing applications". In: *Microelectronics Reliability* 130 (2022), p. 114486. DOI: 10.1016/j.microrel.2022.114486. URL: https://doi.org/10.1016/j.microrel.2022.114486.